

05_simulation

March 27, 2026

1 05 — Scenariosimulering og sårbarhetsmål

Stargate +230/+520 MW scenarioer med fire modeller: **OLS (HAC)**, **OLS (sesong)**, **XGBoost**, **MS-DR**.

ISO-regimeandel (logistisk regresjon) og ekstrempriser.

Input: intermediate/df_processed.parquet, intermediate/df_iso.parquet,
intermediate/models.pkl, intermediate/models_ols_seasonal.pkl,
intermediate/models_xgb.pkl, intermediate/models_msdr.pkl,
intermediate/seasonal_medians.pkl, intermediate/cv_folds.pkl
Output: intermediate/scenarios.pkl

```
[40]: import os, warnings, pickle
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

from src.config import (
    TRAIN_YEARS, TEST_YEARS, INTERMEDIATE_DIR, FIG_DIR,
    DESEASON_TARGET, DESEASON_FEATURES,
    STARGATE_MW, STARGATE_TOTAL_MW, TARGET,
    SCALE_COLS, SCALED_NAME_MAP, SEASON_DUMMIES, SEASON_ORDER,
    apply_style, C1, C2, C3, C4, C5, PALETTE
)
from src.scenario import build_scenario_deseason, fill_group
from src.evaluation import demand_level

apply_style()
```

```
[41]: df = pd.read_parquet(INTERMEDIATE_DIR + "df_processed.parquet")
df_iso = pd.read_parquet(INTERMEDIATE_DIR + "df_iso.parquet")

# OLS (HAC) - baseline modell
```

```

with open(INTERMEDIATE_DIR + "models.pkl", "rb") as f:
    models = pickle.load(f)
ols_model = models["ols_model"]

# OLS (sesong)
try:
    with open(INTERMEDIATE_DIR + "models_ols_seasonal.pkl", "rb") as f:
        ols_seasonal_data = pickle.load(f)
        ols_seasonal_model = ols_seasonal_data["models"]
        SEASONAL_FEATURES = ols_seasonal_data["features"]
        ols_seasonal_sims = ols_seasonal_data["sim_results"]
        print("OLS (sesong): OK")
except FileNotFoundError:
    print("OBS: models_ols_seasonal.pkl ikke funnet")
    ols_seasonal_sims = {}

# XGBoost
try:
    with open(INTERMEDIATE_DIR + "models_xgb.pkl", "rb") as f:
        xgb_data = pickle.load(f)
        xgb_sims = xgb_data["sim_results"]
        print("XGBoost: OK")
except FileNotFoundError:
    print("OBS: models_xgb.pkl ikke funnet")
    xgb_sims = {}

# MS-DR
try:
    with open(INTERMEDIATE_DIR + "models_msdr.pkl", "rb") as f:
        msdr_data = pickle.load(f)
        msdr_sims = msdr_data["sim_results"]
        print("MS-DR: OK")
except FileNotFoundError:
    print("OBS: models_msdr.pkl ikke funnet")
    msdr_sims = {}

# Sesongmedianer og CV-data
with open(INTERMEDIATE_DIR + "seasonal_medians.pkl", "rb") as f:
    med_data = pickle.load(f)

with open(INTERMEDIATE_DIR + "cv_folds.pkl", "rb") as f:
    cv_data = pickle.load(f)
cv_coef_df = cv_data["cv_coef_df"]

# Etterspørselsterskler
train_cons = df_iso.loc[df_iso.index.year.isin(TRAIN_YEARS), "cons_N04"]
t1, t2 = train_cons.quantile([1/3, 2/3])

```

```
print(f"\ndf: {len(df):,}, df_iso: {len(df_iso):,} ISO-timer")
```

OLS (sesong): OK

XGBoost: OK

MS-DR: OK

df: 53,394, df_iso: 34,916 ISO-timer

1.1 Seksjon 10 — Scenariosimulering

```
[42]: SEC10_DIR = FIG_DIR + "section_10/"
os.makedirs(SEC10_DIR, exist_ok=True)

# OLS (HAC): beregn ΔP direkte (lineær + konstant effekt)
X_base = df_iso[DESEASON_FEATURES]
X_base_c = sm.add_constant(X_base)

MODEL_DP = {
    "OLS (HAC)": "dP_OLS",
    "OLS (sesong)": "dP_OLS_seasonal",
    "XGBoost": "dP_XGB",
    "MS-DR": "dP_MSDR",
}

scenarios = {}
for delta, label in [(STARGATE_MW, f"Fase 1 (+{STARGATE_MW} MW)"),
                     (STARGATE_TOTAL_MW, f"Fase 1+2 (+{STARGATE_TOTAL_MW}
→MW)"]):
    # OLS (HAC): konstant ΔP
    X_scen = build_scenario_deseason(X_base, delta)
    X_scen_c = sm.add_constant(X_scen)
    dP_ols = ols_model.predict(X_scen_c) - ols_model.predict(X_base_c)

    result = pd.DataFrame(index=df_iso.index)
    result["season"] = df_iso["season"].values
    result["cons_NO4"] = df_iso["cons_NO4"].values
    result["demand_level"] = df_iso["cons_NO4"].map(lambda c: demand_level(c,
→t1, t2))

    # Samle ΔP fra alle modeller
    result["dP_OLS"] = dP_ols
    result["dP_OLS_seasonal"] = ols_seasonal_sims[delta]["dP"].values
    result["dP_XGB"] = xgb_sims[delta]["dP"].values
    result["dP_MSDR"] = msdr_sims[delta]["dP"].values
```

```

# Scenariopriser per modell: faktisk pris + ΔP
for col in MODEL_DP.values():
    result[f"P_scen_{col}"] = df_iso[TARGET].values + result[col].values

scenarios[delta] = result
print(f"{label}:")
for name, col in MODEL_DP.items():
    print(f"  {name:24s} snitt ΔP = {result[col].mean():+.1f} NOK/MWh")

print("\nScenarier beregnet.")

```

Fase 1 (+230 MW):

OLS (HAC)	snitt ΔP = +70.4 NOK/MWh
OLS (sesong)	snitt ΔP = +70.0 NOK/MWh
XGBoost	snitt ΔP = +74.1 NOK/MWh
MS-DR	snitt ΔP = +56.6 NOK/MWh

Fase 1+2 (+520 MW):

OLS (HAC)	snitt ΔP = +159.1 NOK/MWh
OLS (sesong)	snitt ΔP = +158.2 NOK/MWh
XGBoost	snitt ΔP = +139.3 NOK/MWh
MS-DR	snitt ΔP = +128.0 NOK/MWh

Scenarier beregnet.

```

[43]: # Tabell 1: ΔP totalt og per sesong
for delta, label in [(STARGATE_MW, f"Fase 1 (+{STARGATE_MW} MW)"),
                    (STARGATE_TOTAL_MW, f"Fase 1+2 (+{STARGATE_TOTAL_MW}
↪MW)"))]:
    res = scenarios[delta]
    print("=" * 70)
    print(f"ΔP (NOK/MWh) - {label}")
    print("=" * 70)

    rows = []
    for name, col in MODEL_DP.items():
        row = {"Modell": name, "Total": round(res[col].mean(), 1)}
        for s_en, s_no in SEASON_ORDER.items():
            mask = res["season"] == s_en
            row[s_no] = round(res.loc[mask, col].mean(), 1)
        rows.append(row)

    display(pd.DataFrame(rows).set_index("Modell").style.format("{:.1f}"))
    print()

```

```

=====
ΔP (NOK/MWh) - Fase 1 (+230 MW)
=====

```

<pandas.io.formats.style.Styler at 0x17615ee90>

```
=====
ΔP (NOK/MWh) - Fase 1+2 (+520 MW)
=====
```

```
<pandas.io.formats.style.Styler at 0x17581c510>
```

```
[44]: # Tabell 2: ΔP per etterspørselsnivå
for delta, label in [(STARGATE_MW, f"+{STARGATE_MW} MW"),
                     (STARGATE_TOTAL_MW, f"+{STARGATE_TOTAL_MW} MW")]:
    res = scenarios[delta]
    print(f"ΔP per etterspørselsnivå - {label}")
    rows = []
    for level in ["Lav", "Normal", "Høy"]:
        mask = res["demand_level"] == level
        row = {"Etterspørsel": level}
        for name, col in MODEL_DP.items():
            row[name] = round(res.loc[mask, col].mean(), 1)
        rows.append(row)
    display(pd.DataFrame(rows).set_index("Etterspørsel").style.format("{:.1f}"))
    print()
```

```
ΔP per etterspørselsnivå - +230 MW
```

```
<pandas.io.formats.style.Styler at 0x1761183d0>
```

```
ΔP per etterspørselsnivå - +520 MW
```

```
<pandas.io.formats.style.Styler at 0x1761f77d0>
```

```
[45]: # ΔP-intervall fra HAC SE og CV-folder (OLS baseline)
beta_cons = ols_model.params["cons_avvik"]
se_cons = ols_model.bse["cons_avvik"]
cv_beta_min = cv_coef_df["cons_avvik"].min()
cv_beta_max = cv_coef_df["cons_avvik"].max()

print("=" * 70)
print("ΔP-INTERVALL (OLS HAC)")
print("=" * 70)
for delta, label in [(STARGATE_MW, f"+{STARGATE_MW} MW"),
                     (STARGATE_TOTAL_MW, f"+{STARGATE_TOTAL_MW} MW")]:
    dp = beta_cons * delta
    ci_lo = (beta_cons - 1.96 * se_cons) * delta
    ci_hi = (beta_cons + 1.96 * se_cons) * delta
    cv_lo = cv_beta_min * delta
    cv_hi = cv_beta_max * delta
```

```

print(f"\n {label}:")
print(f"    OLS:  $\Delta P$  = {dp:+.1f} NOK/MWh 95% KI [{ci_lo:+.1f}, {ci_hi:+.1f}]\n")
print(f"    CV-intervall: [{cv_lo:+.1f}, {cv_hi:+.1f}] NOK/MWh")

print(f"\n{'=' * 70}")
print("MODELLSPENN (snitt  $\Delta P$  over alle fire modeller)")
print('=' * 70)
for delta, label in [(STARGATE_MW, f"+{STARGATE_MW} MW"),
                     (STARGATE_TOTAL_MW, f"+{STARGATE_TOTAL_MW} MW")]:
    res = scenarios[delta]
    dp_means = {name: res[col].mean() for name, col in MODEL_DP.items()}
    print(f"\n {label}: [{min(dp_means.values()):+.1f}, {max(dp_means.values()):+.1f}] NOK/MWh")

```

```

=====
 $\Delta P$ -INTERVALL (OLS HAC)
=====

```

```

+230 MW:
    OLS:  $\Delta P$  = +70.4 NOK/MWh 95% KI [+54.7, +86.1]
    CV-intervall: [+14.8, +70.4] NOK/MWh

+520 MW:
    OLS:  $\Delta P$  = +159.1 NOK/MWh 95% KI [+123.6, +194.6]
    CV-intervall: [+33.4, +159.1] NOK/MWh

```

```

=====
MODELLSPENN (snitt  $\Delta P$  over alle fire modeller)
=====

```

```

+230 MW: [+56.6, +74.1] NOK/MWh

+520 MW: [+128.0, +159.1] NOK/MWh

```

1.2 Seksjon 11 — Sårbarhetsmål

```

[46]: # Mål 1: ISO-regimeandel (logistisk regresjon)
      # Bruker StandardScaler-tilnærmingen (ikke sesongjustert)
      SEC11_DIR = FIG_DIR + "section_11/"
      os.makedirs(SEC11_DIR, exist_ok=True)

      df["is_iso"] = (df["regime_group"] == "ISO").astype(int)

      # Fit scaler for logit
      scaler = StandardScaler()

```

```

scaler.fit(df.loc[df.index.year.isin(TRAIN_YEARS), SCALE_COLS])
scaled = scaler.transform(df[SCALE_COLS])
for i, col in enumerate(SCALE_COLS):
    df[SCALED_NAME_MAP[col]] = scaled[:, i]

for s in SEASON_DUMMIES:
    df[s] = (df["season"] == s).astype(int)

LOGIT_FEATURES = ["cons_c", "fill_c"] + SEASON_DUMMIES
df_logit_train = df[df.index.year.isin(TRAIN_YEARS)]

logit_model = LogisticRegression(max_iter=1000, random_state=42)
logit_model.fit(df_logit_train[LOGIT_FEATURES], df_logit_train["is_iso"])

baseline_iso_share = df["is_iso"].mean()
print(f"Faktisk ISO-andel: {baseline_iso_share*100:.1f}%")

iso_share_results = []
for delta, label in [(0, "Baseline"), (STARGATE_MW, f"+{STARGATE_MW} MW"),
                     (STARGATE_TOTAL_MW, f"+{STARGATE_TOTAL_MW} MW")]:
    df_scen = df.copy()
    if delta > 0:
        df_scen["cons_N04"] = df_scen["cons_N04"] + delta
        if "net_position" in df_scen.columns:
            df_scen["net_position"] = df_scen["prod_total_N04"] -
df_scen["cons_N04"]
        scaled_scen = scaler.transform(df_scen[SCALE_COLS])
        for i, col in enumerate(SCALE_COLS):
            df_scen[SCALED_NAME_MAP[col]] = scaled_scen[:, i]

    p_iso = logit_model.predict_proba(df_scen[LOGIT_FEATURES])[:, 1]
    row = {"Scenario": label, "Total": round(p_iso.mean() * 100, 1)}
    for s_en, s_no in SEASON_ORDER.items():
        mask = df_scen["season"] == s_en
        row[s_no] = round(p_iso[mask].mean() * 100, 1)
    iso_share_results.append(row)

iso_share_df = pd.DataFrame(iso_share_results).set_index("Scenario")
print("\nPredikert ISO-andel (%):")
display(iso_share_df.style.format("{:.1f}"))

```

Faktisk ISO-andel: 65.4%

Predikert ISO-andel (%):

<pandas.io.formats.style.Styler at 0x17628c850>

```
[47]: # Mål 2: Ekstrempriser (P > 90. persentil)
p90_threshold = np.percentile(df_iso[TARGET], 90)
fill_median = df_iso["fill_rate"].median()

print(f"P90-terskel: {p90_threshold:.1f} NOK/MWh")

# Beregn for hver modell
for model_name, dp_col in MODEL_DP.items():
    print(f"\n{'=' * 70}")
    print(f"Andel timer med pris > P90 - {model_name}")
    print(f"{'=' * 70}")

    extreme_rows = []
    for delta, label in [(0, "Baseline"), (STARGATE_MW, f"+{STARGATE_MW} MW"),
                        (STARGATE_TOTAL_MW, f"+{STARGATE_TOTAL_MW} MW")]:
        if delta == 0:
            prices = df_iso[TARGET]
        else:
            prices = scenarios[delta][f"P_scen_{dp_col}"]

        pct = 100 * (prices > p90_threshold).sum() / len(prices)
        row = {"Scenario": label, "Total (%)": round(pct, 1)}

        for s_en, s_no in SEASON_ORDER.items():
            mask = df_iso["season"] == s_en
            p_sub = prices[mask] if delta == 0 else prices.iloc[mask.values]
            row[s_no] = round(100 * (p_sub > p90_threshold).sum() / len(p_sub), 1)
        ↪1) if len(p_sub) > 0 else 0

        extreme_rows.append(row)

    display(pd.DataFrame(extreme_rows).set_index("Scenario"))
```

P90-terskel: 543.5 NOK/MWh

```
=====
Andel timer med pris > P90 - OLS (HAC)
=====
```

	Total (%)	Vinter	Vår	Sommer	Høst
Scenario					
Baseline	10.0	19.6	8.4	1.4	9.5
+230 MW	15.4	28.3	12.7	3.1	16.0
+520 MW	22.9	41.2	21.6	6.0	20.8

```
=====
Andel timer med pris > P90 - OLS (sesong)
=====
```


	Total (%)	Vinter	Vår	Sommer	Høst
Scenario					
Baseline	10.0	19.6	8.4	1.4	9.5
+230 MW	18.6	44.1	13.4	0.8	13.2
+520 MW	32.5	81.9	24.4	0.4	17.5

=====

Andel timer med pris > P90 - XGBoost

=====

	Total (%)	Vinter	Vår	Sommer	Høst
Scenario					
Baseline	10.0	19.6	8.4	1.4	9.5
+230 MW	17.4	37.2	14.1	3.1	13.0
+520 MW	24.5	49.9	18.4	8.6	17.9

=====

Andel timer med pris > P90 - MS-DR

=====

	Total (%)	Vinter	Vår	Sommer	Høst
Scenario					
Baseline	10.0	19.6	8.4	1.4	9.5
+230 MW	15.1	28.8	11.6	2.6	15.7
+520 MW	21.0	40.6	17.9	4.1	19.4

1.3 Seksjon 12 — MS-DR: Regime-spesifikke forbrukskoeffisienter

$\hat{Q}$ per regime med 95 % KI, steady-state sannsynligheter og vektet ΔP .

```
[48]: import os
SEC12_DIR = f"{FIG_DIR}section_12/"
os.makedirs(SEC12_DIR, exist_ok=True)

msdr_result = msdr_data["result"]

# Regime-spesifikke  $\hat{Q}$  ( $x_1$  = cons_avvik,  $x_2$  = fill_avvik,  $x_3$  = wind_c)
_b0 = msdr_result.params["x1[0]"]
_b1 = msdr_result.params["x1[1]"]
_ci = msdr_result.conf_int(alpha=0.05)
_ci0 = (_ci.loc["x1[0]", 0], _ci.loc["x1[0]", 1])
_ci1 = (_ci.loc["x1[1]", 0], _ci.loc["x1[1]", 1])

# Steady-state fra overgangsparametere (unambiguous)
_p00 = msdr_result.params["p[0->0]"]
_p10 = msdr_result.params["p[1->0]"] # P(1→0)
```

```

_p01 = 1 - _p00                                # P(0→1)
_pi0 = _p10 / (_p01 + _p10)                    # stasjonær sannsynlighet regime 0
_pi1 = _p01 / (_p01 + _p10)                    # stasjonær sannsynlighet regime 1

_dP0 = _b0 * STARGATE_MW
_dP1 = _b1 * STARGATE_MW
_dP_w = (_pi0 * _b0 + _pi1 * _b1) * STARGATE_MW

# 95 % KI for vektet ΔP (pi fast, usikkerhet fra koeffisient-KI)
_dP_w_lo = (_pi0 * _ci0[0] + _pi1 * _ci1[0]) * STARGATE_MW
_dP_w_hi = (_pi0 * _ci0[1] + _pi1 * _ci1[1]) * STARGATE_MW

print("MS-DR: Regime-spesifikke forbrukskoeffisienter")
print("=" * 65)
print(f"Regime 0 (lav volatilitet):  = {_b0:.5f}  "
      f"KI [{_ci0[0]:.5f}, {_ci0[1]:.5f}]")
print(f" → ΔP (+{STARGATE_MW} MW) = {_dP0:.1f}  "
      f"KI [{_ci0[0]*STARGATE_MW:.1f}, {_ci0[1]*STARGATE_MW:.1f}] NOK/MWh")
print(f"Regime 1 (høy volatilitet):  = {_b1:.5f}  "
      f"KI [{_ci1[0]:.5f}, {_ci1[1]:.5f}]")
print(f" → ΔP (+{STARGATE_MW} MW) = {_dP1:.1f}  "
      f"KI [{_ci1[0]*STARGATE_MW:.1f}, {_ci1[1]*STARGATE_MW:.1f}] NOK/MWh")
print(f"\nSteady-state:  = {_pi0:.3f},  = {_pi1:.3f}")
print(f"Vektet ΔP = {_dP_w:.1f} NOK/MWh  "
      f"KI [{_dP_w_lo:.1f}, {_dP_w_hi:.1f}]")
print(f"(Kontroll mot lagret sim-snitt:  "
      f"{msdr_sims[STARGATE_MW]['dP'].mean():.1f} NOK/MWh)")

fig, ax = plt.subplots(figsize=(8, 5))
_regs = ["Regime 0 (lav vol.)", "Regime 1 (høy vol.)"]
_betas = [_b0, _b1]
_cis = [_ci0, _ci1]
_rcols = [C1, C2]

for i, (reg, b, ci, col) in enumerate(zip(_regs, _betas, _cis, _rcols)):
    ax.bar(i, b * STARGATE_MW, color=col, alpha=0.85, width=0.5)
    ax.errorbar(
        i, b * STARGATE_MW,
        yerr=[(b - ci[0]) * STARGATE_MW, [(ci[1] - b) * STARGATE_MW]],
        fmt="none", color="black", capsize=10, lw=2
    )

ax.axhline(_dP_w, color=C4, ls="--", lw=2,
           label=f"Vektet snitt ( ={_pi0:.2f}, ={_pi1:.2f}): {_dP_w:.1f} NOK/
           ↪MWh")
ax.set_xticks([0, 1])
ax.set_xticklabels(_regs)

```

```

ax.set_ylabel("Estimert ΔP (NOK/MWh) ved +230 MW")
ax.set_title("MS-DR: Forbrukseffekt per regime med 95 % KI")
ax.legend(fontsize=12)
plt.tight_layout()
plt.savefig(f"{SEC12_DIR}msdr_koeff_per_regime.pdf", bbox_inches="tight")
plt.show()

```

MS-DR: Regime-spesifikke forbrukskoeffisienter

=====

Regime 0 (lav volatilitet): = 0.07271 KI [0.06522, 0.08019]

→ ΔP (+230 MW) = 16.7 KI [15.0, 18.4] NOK/MWh

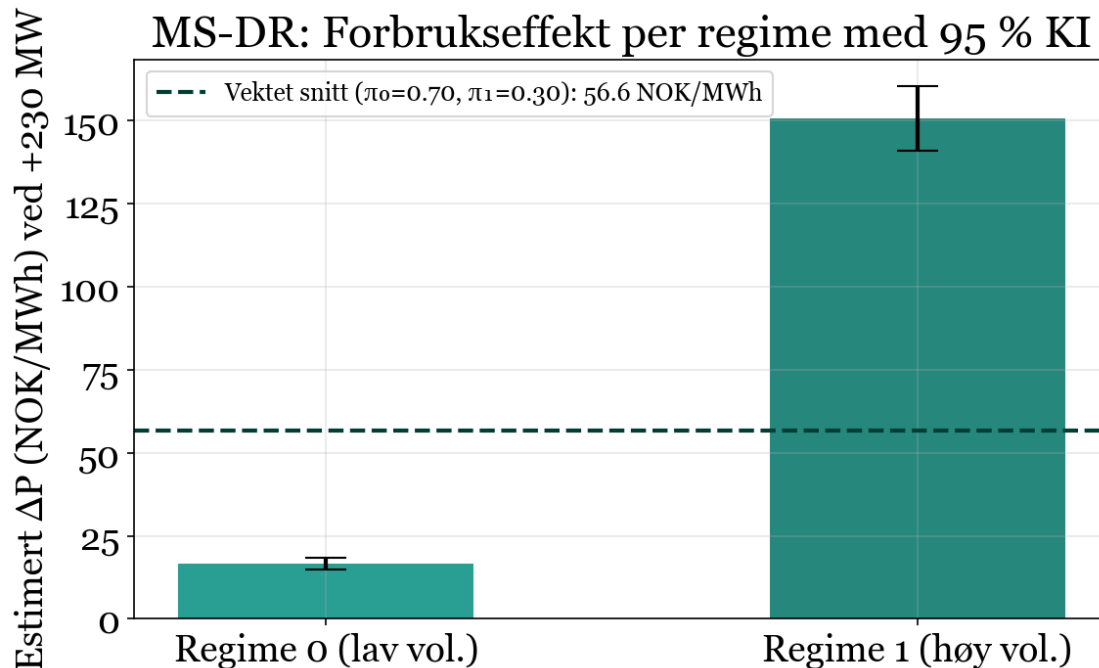
Regime 1 (høy volatilitet): = 0.65543 KI [0.61293, 0.69792]

→ ΔP (+230 MW) = 150.7 KI [141.0, 160.5] NOK/MWh

Steady-state: = 0.702, = 0.298

Vektet ΔP = 56.6 NOK/MWh KI [52.5, 60.8]

(Kontroll mot lagret sim-snitt: 56.6 NOK/MWh)



1.4 Seksjon 14 — Konvergenstabell (alle modeller)

Sammenstilling av estimert prisseffekt (+230 MW) på tvers av alle fire modeller med tilhørende usikkerhetsmål. Konvergens mellom modellene er sterk evidens for robusthet.

```

[49]: SEC14_DIR = f"{FIG_DIR}section_14/"
os.makedirs(SEC14_DIR, exist_ok=True)

_bc = ols_model.params["cons_avvik"]
_se_c = ols_model.bse["cons_avvik"]

_conv_rows = [
    {
        "Modell": "OLS (HAC)",
        "ΔP (NOK/MWh)": round(_bc * STARGATE_MW, 1),
        "KI / spenn": f"[_bc - 1.96*_se_c]*STARGATE_MW:.1f}, "
        f"[_bc + 1.96*_se_c]*STARGATE_MW:.1f}]",
        "Usikkerhetstype": "95 % HAC KI",
    },
    {
        "Modell": "OLS (sesong)",
        "ΔP (NOK/MWh)": round(ols_seasonal_sims[STARGATE_MW]["dP"].mean(), 1),
        "KI / spenn": (
            f"Vinter: {ols_seasonal_sims[STARGATE_MW].loc[ols_seasonal_sims[STARGATE_MW]['season']=='Winter','dP'].mean():.0f} - "
            f"Sommer: {ols_seasonal_sims[STARGATE_MW].loc[ols_seasonal_sims[STARGATE_MW]['season']=='Summer','dP'].mean():.0f}"
        ),
        "Usikkerhetstype": "Sesongspenn",
    },
    {
        "Modell": "XGBoost",
        "ΔP (NOK/MWh)": round(scenarios[STARGATE_MW]["dP_XGB"].mean(), 1),
        "KI / spenn": (
            f"Vinter: {scenarios[STARGATE_MW].loc[scenarios[STARGATE_MW]['season']=='Winter','dP_XGB'].mean():.0f} - "
            f"Sommer: {scenarios[STARGATE_MW].loc[scenarios[STARGATE_MW]['season']=='Summer','dP_XGB'].mean():.0f}"
        ),
        "Usikkerhetstype": "Sesongspenn",
    },
    {
        "Modell": "MS-DR (vektet)",
        "ΔP (NOK/MWh)": round(_dP_w, 1),
        "KI / spenn": f"[_dP_w_lo:.1f}, {_dP_w_hi:.1f}]",
        "Usikkerhetstype": "Regime-vektet 95 % KI",
    },
]

conv_df = pd.DataFrame(_conv_rows).set_index("Modell")
print("Konvergenstabell - Estimert priseffekt av +230 MW Stargate-lastøkning")

```

```

print("=" * 70)
display(conv_df)

# Lagre
conv_df.to_csv(f"{INTERMEDIATE_DIR}convergence_table.csv")
print(f"\nLagret: {INTERMEDIATE_DIR}convergence_table.csv")

# Visuelt: punktestimater med usikkerhetsintervaller
fig, ax = plt.subplots(figsize=(10, 5))

_models_conv = conv_df.index.tolist()
_dp_vals_conv = conv_df["ΔP (NOK/MWh)"].values.astype(float)

# Plott punktestimater
for i, (mod, dp) in enumerate(zip(_models_conv, _dp_vals_conv)):
    ax.scatter(dp, i, s=100, color=PALETTE[i % len(PALETTE)], zorder=5)

# Horisontale linjer
for i in range(len(_models_conv)):
    ax.axhline(i, color="grey", lw=0.5, ls=":", zorder=1)

ax.axvline(np.mean(_dp_vals_conv), color=C4, ls="--", lw=1.5,
           label=f"Snitt på tvers: {np.mean(_dp_vals_conv):.1f} NOK/MWh")

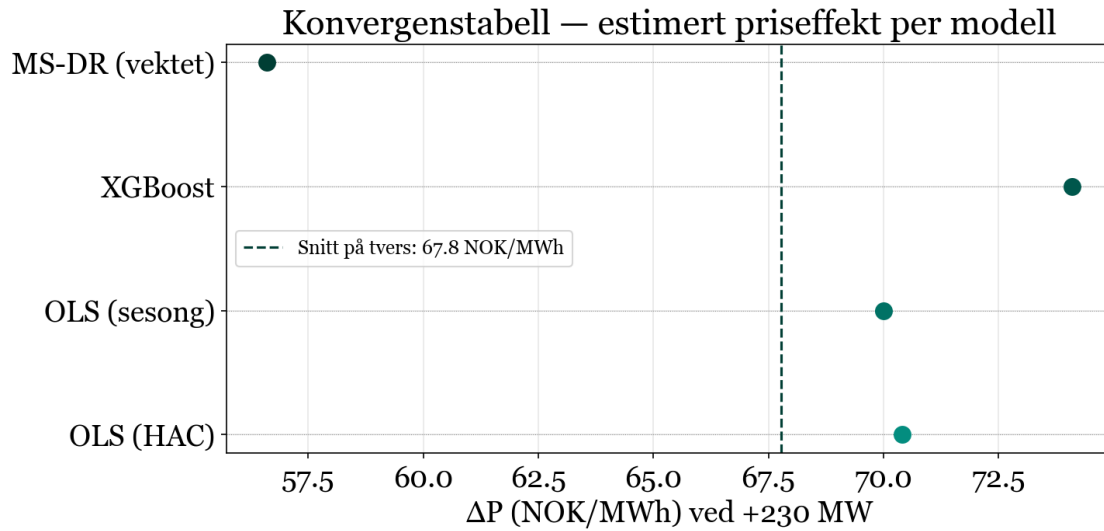
ax.set_yticks(range(len(_models_conv)))
ax.set_yticklabels(_models_conv)
ax.set_xlabel("ΔP (NOK/MWh) ved +230 MW")
ax.set_title("Konvergenstabell - estimert priseffekt per modell")
ax.legend(fontsize=12)
plt.tight_layout()
plt.savefig(f"{SEC14_DIR}convergence_plot.pdf", bbox_inches="tight")
plt.show()

```

Konvergenstabell - Estimert priseffekt av +230 MW Stargate-lastøkning

	ΔP (NOK/MWh)	KI / spenn	Usikkerhetstype
Modell			
OLS (HAC)	70.400000	[54.7, 86.1]	95 % HAC KI
OLS (sesong)	70.000000	Vinter: 179 - Sommer: -32	Sesongspenn
XGBoost	74.099998	Vinter: 126 - Sommer: 47	Sesongspenn
MS-DR (vektet)	56.600000	[52.5, 60.8]	Regime-vektet 95 % KI

Lagret: intermediate/convergence_table.csv



```
[50]: # Lagre
with open(INTERMEDIATE_DIR + "scenarios.pkl", "wb") as f:
    pickle.dump({
        "scenarios": scenarios,
        "MODEL_DP": MODEL_DP,
        "iso_share_results": iso_share_results,
        "p90_threshold": p90_threshold,
        "cv_beta_min": cv_coef_df["cons_avvik"].min(),
        "cv_beta_max": cv_coef_df["cons_avvik"].max(),
    }, f)

print("Lagret scenarios.pkl")
```

Lagret scenarios.pkl